

# 1. Methodology

## 1.1 Software Development Methodology

The development of LeaveSync followed an **Agile iterative-incremental development methodology**. Agile was selected because the proposed system contains several interconnected features, particularly authentication, multi-factor authentication, device fingerprinting, leave validation, approval routing, and digital signature verification. Developing these features incrementally allowed the researchers to implement, test, evaluate, and improve individual system components before integrating them into the complete system.

The development process consisted of the following recurring activities:

**Requirements Gathering and Analysis.** The researchers identified the functional and security requirements of the system based on the existing leave management process. The requirements included user authentication, multi-factor authentication, device verification, leave request submission, leave balance validation, approval routing, and digital signature-based approval.

**System Design.** The identified requirements were translated into system designs, database structures, process models, and interface designs. BPMN and UML Activity Diagrams were used to represent the existing and proposed processes.

**Implementation.** The researchers developed the LeaveSync web application using PHP for the backend, MySQL for data storage, and HTML, CSS, and JavaScript for the frontend. Security-related functions were implemented using appropriate authentication, device fingerprinting, and cryptographic technologies.

**Testing and Evaluation.** Individual system functions were tested to determine whether they operated according to the specified requirements. The researchers also performed functional testing of the complete workflow, including login, multi-factor authentication, trusted-device verification, leave submission, supervisor approval, HR approval, and leave balance updating.

**Review and Refinement.** Errors and inconsistencies identified during testing were corrected and the affected components were refined. The development cycle was repeated until the major system functions operated according to the identified requirements.

The iterative process allowed the researchers to progressively develop the system through the following major stages:

**Authentication → Multi-Factor Authentication → Device Fingerprinting → Leave Request → Leave Validation → Approval Routing → Digital Signature Verification → Leave Balance Update → Audit Logging**

## 1.2 Scrum Team and Responsibilities

The development of LeaveSync followed Scrum practices within the Agile methodology. The Scrum team consisted of the researchers who collaborated in planning, designing, developing, testing, and refining the system. The team adopted the responsibilities of the Product Owner, Scrum Master, and Development Team to organize the development activities and ensure that the system requirements were addressed.

| Scrum Role           | Responsibilities                                                                                                                                                                                                                                                                               |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Product Owner</b> | Identified and prioritized the system requirements, maintained the Product Backlog, clarified the expected system features, and ensured that the developed functions addressed the needs of the intended users and the objectives of the study.                                                |
| <b>Scrum Master</b>  | Facilitated the Agile development process, monitored sprint activities, helped identify and resolve development issues, coordinated team activities, and ensured that the team followed the agreed development workflow.                                                                       |
| <b>Programmer</b>    | Designed, programmed, tested, integrated, and documented the LeaveSync system. The team was responsible for implementing the database, user interface, authentication, device fingerprinting, leave processing, approval workflow, digital signature verification, and audit logging features. |

The Scrum roles were used to establish clear responsibilities among the researchers while maintaining collaboration throughout the development process. Although the team members worked collaboratively, assigning these responsibilities helped organize decision-making, sprint activities, and system development tasks.

## 1.3 Requirements Gathering

Requirements gathering was conducted to identify the functional, operational, and security requirements of the proposed LeaveSync system. The researchers gathered information about the existing leave management process and used the findings as the basis for determining the features and processes that needed to be included in the proposed system.

The following methods were used to gather the system requirements:

**Interview.** Interviews were conducted with personnel involved in the leave management process to determine how leave requests are currently submitted, reviewed, approved, and recorded. The interviews also helped identify difficulties encountered in the current process and the requirements that could improve leave management.

**Observation.** The researchers observed the existing leave request process to understand the actual sequence of activities performed by employees, supervisors, and HR personnel. Particular attention was given to leave submission, notification, approval, manual encoding, and leave balance updating.

**Document Review.** Existing leave-related forms, procedures, and records were reviewed to identify the information required in a leave request and the manner in which approved and rejected requests are recorded.

The gathered requirements were analyzed and converted into system features and tasks. These requirements were then prioritized and organized in the Product Backlog, which served as the basis for planning the development sprints.

## 1.4 Product Backlog

The Product Backlog contained the prioritized features and development requirements identified for LeaveSync. The backlog was used to organize the system requirements and determine which features would be implemented during each development sprint. Items were prioritized according to their importance to the core leave management process, security requirements, and overall system functionality.

| Priority | Product Backlog Item        | Description                                                                                   |
|----------|-----------------------------|-----------------------------------------------------------------------------------------------|
| 1        | User Authentication         | Allow authorized users to securely access the system using their organization account.        |
| 2        | Multi-Factor Authentication | Require an additional authentication factor to strengthen account security.                   |
| 3        | User and Role Management    | Manage employee, supervisor, HR, and administrator accounts and permissions.                  |
| 4        | Device Fingerprinting       | Identify and verify the device used by an employee when accessing protected system functions. |
| 5        | Leave Type Management       | Allow authorized personnel to manage available leave types and their corresponding rules.     |

|    |                                      |                                                                                                   |
|----|--------------------------------------|---------------------------------------------------------------------------------------------------|
| 6  | Leave Balance Management             | Maintain employee leave balances and track available, pending, and used leave days.               |
| 7  | Leave Request Submission             | Allow employees to submit leave requests with the required information.                           |
| 8  | Leave Request Validation             | Validate dates, weekdays, leave balance, and overlapping requests before submission.              |
| 9  | Approval Workflow                    | Route leave requests to the appropriate supervisor and HR personnel based on the employee's role. |
| 10 | Digital Signature / Passkey Approval | Require authorized approvers to verify their approval using a WebAuthn/FIDO2 passkey.             |
| 11 | Leave Balance Updating               | Automatically reserve, restore, or transfer leave days based on the request decision.             |
| 12 | Audit Logging                        | Record important system actions, including approval and rejection activities.                     |
| 13 | Leave Request Monitoring             | Allow employees and authorized personnel to monitor the status of submitted requests.             |
| 14 | Notifications                        | Provide users with information regarding leave request status and approval activities.            |
| 15 | System Administration                | Provide administrators with functions for maintaining system settings and records.                |

The Product Backlog was reviewed and refined throughout the development process. Higher-priority items related to authentication, leave processing, approval, and security were implemented first because they represented the core functions of the LeaveSync system.

## 1.5 Sprint Plan

The development of **LeaveSync** followed a **Lightweight Scrum approach** organized into iterative and incremental sprints. The development process began with a **Product Backlog** containing the identified system requirements and user stories. At the beginning of each sprint, the researchers selected and prioritized backlog items based on their importance and the current development needs. These selected items formed the **Sprint Backlog**.

During each sprint, the researchers performed development, integration, testing, and refinement of the selected backlog items. Daily progress and issues were monitored throughout the sprint. At the end of each sprint, the completed work was demonstrated and reviewed, feedback was

considered, and a retrospective was conducted to identify improvements for the succeeding sprint. Completed and tested features were added to the working **LeaveSync increment**.

| <b>Sprint</b>   | <b>Sprint Activities</b>                                                                                                                                                    | <b>Expected Increment</b>                                             |
|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| <b>Sprint 1</b> | Product Backlog preparation, requirements clarification, user stories, system architecture, database structure, and initial system setup                                    | Initial working LeaveSync foundation with prioritized Product Backlog |
| <b>Sprint 2</b> | Development of prioritized authentication, user-role, and account-management user stories; testing and integration                                                          | Working authentication and user-role features                         |
| <b>Sprint 3</b> | Development and integration of prioritized device fingerprinting, trusted-device, and security-related user stories; testing and refinement                                 | Working device identification and trusted-device verification         |
| <b>Sprint 4</b> | Development of prioritized leave-management user stories, including leave types, leave balances, leave requests, date validation, weekday calculation, and overlap checking | Working leave-management features integrated with previous increments |
| <b>Sprint 5</b> | Development and integration of prioritized approval, leave reservation, supervisor approval, HR approval, and role-based routing user stories                               | Working leave approval workflow                                       |
| <b>Sprint 6</b> | Development and integration of prioritized WebAuthn/FIDO2 passkey verification and digital signature-based approval user stories                                            | Working digitally verified approval features                          |
| <b>Sprint 7</b> | Integration and refinement of completed features, audit logging, balance updates, security checks,                                                                          | More complete and tested LeaveSync increment                          |

|                 |                                                                                                                     |                                                                    |
|-----------------|---------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------|
|                 | functional testing, and defect correction based on feedback                                                         |                                                                    |
| <b>Sprint 8</b> | Final backlog review, remaining improvements, system-wide testing, deployment, documentation, and system evaluation | Final working LeaveSync system and completed project documentation |

The sprint activities were adjusted when necessary based on development progress and issues identified during testing. This iterative approach allowed the researchers to refine individual components before integrating them into the complete LeaveSync system.

## 1.6 Work Breakdown Structure (WBS)

The Work Breakdown Structure (WBS) was used to divide the **LeaveSync** project into manageable sprint activities, tasks, and deliverables. The WBS followed the project's **Lightweight Scrum approach**, where development was organized into iterative sprints. Each sprint included sprint planning, implementation of selected Product Backlog items, testing, Sprint Review, Sprint Retrospective, and backlog refinement. The completed and tested work from each sprint contributed to the working system increment.

## 1.0 LeaveSync Project

### 1.1 Sprint 1 – Project Preparation and Initial System Foundation

#### 1.1.1 Sprint Planning and Preparation

- **1.1.1.1 Conduct Sprint Planning**
  - Define the Sprint Goal and select prioritized Product Backlog items
  - **Role:** Scrum Master, Product Owner, Development Team
- **1.1.1.2 Define Acceptance Criteria**
  - Define the conditions for completing the selected requirements
  - **Role:** Product Owner, Development Team
- **1.1.1.3 Finalize Sprint Backlog**
  - Organize selected user stories and development tasks
  - **Role:** Development Team

#### 1.1.2 Requirements and Existing System Analysis

- **1.1.2.1 Identify Existing Leave Management Process**

- **1.1.2.2 Conduct Interviews**
- **1.1.2.3 Observe the Existing Process**
- **1.1.2.4 Review Existing Documents and Forms**
- **1.1.2.5 Identify Functional and Security Requirements**
- **Role:** Product Owner, Development Team

### **1.1.3 Initial System Design**

- **1.1.3.1 Define User Roles**
- **1.1.3.2 Design Initial System Architecture**
- **1.1.3.3 Design Initial Database Structure**
- **1.1.3.4 Prepare Initial User Interface Design**
- **Role:** Development Team

### **1.1.4 Sprint Review and Retrospective**

- **1.1.4.1 Conduct Sprint Review**
  - Present the initial system foundation and gather feedback
  - **Role:** Development Team, Scrum Master, Product Owner
- **1.1.4.2 Conduct Sprint Retrospective**
  - Identify improvements for the next sprint
  - **Role:** Scrum Master, Development Team
- **1.1.4.3 Refine Product Backlog**
  - Update and reprioritize backlog items based on feedback
  - **Role:** Product Owner, Development Team

**Expected Increment:** Approved requirements, initial system structure, database foundation, and prioritized Product Backlog.

## **1.2 Sprint 2 – User Authentication and Role Management**

### **1.2.1 Sprint Planning and Preparation**

- **1.2.1.1 Conduct Sprint Planning**
- **1.2.1.2 Define Acceptance Criteria**
- **1.2.1.3 Finalize Sprint Backlog**

### **1.2.2 Authentication User Stories**

- **1.2.2.1 Implement User Authentication**
- **1.2.2.2 Implement Google Account Authentication**

- 1.2.2.3 Restrict Authorized Institutional Accounts
- 1.2.2.4 Implement User Role Management
- 1.2.2.5 Implement Multi-Factor Authentication

### **1.2.3 Testing and Integration**

- 1.2.3.1 Test Authentication
- 1.2.3.2 Test User Role Access
- 1.2.3.3 Test Multi-Factor Authentication
- 1.2.3.4 Correct Identified Defects
- 1.2.3.5 Integrate Tested Features with Existing Increment

### **1.2.4 Sprint Review and Retrospective**

- 1.2.4.1 Conduct Sprint Review
- 1.2.4.2 Gather Stakeholder Feedback
- 1.2.4.3 Conduct Sprint Retrospective
- 1.2.4.4 Refine Product Backlog

**Expected Increment:** Working authentication, user-role management, and MFA features integrated into LeaveSync.

## **1.3 Sprint 3 – Device Fingerprinting and Trusted-Device Verification**

### **1.3.1 Sprint Planning and Preparation**

- 1.3.1.1 Conduct Sprint Planning
- 1.3.1.2 Define Acceptance Criteria
- 1.3.1.3 Finalize Sprint Backlog

### **1.3.2 Device Security User Stories**

- 1.3.2.1 Implement Device Fingerprinting
- 1.3.2.2 Generate and Store Device Identifier
- 1.3.2.3 Implement Trusted-Device Registration
- 1.3.2.4 Implement Trusted-Device Verification
- 1.3.2.5 Implement Unrecognized-Device Handling

### **1.3.3 Testing and Integration**

- 1.3.3.1 Test Device Identification

- 1.3.3.2 Test Trusted-Device Verification
- 1.3.3.3 Test Authentication and Device Verification Integration
- 1.3.3.4 Correct Identified Defects

### **1.3.4 Sprint Review and Retrospective**

- 1.3.4.1 Conduct Sprint Review
- 1.3.4.2 Gather Stakeholder Feedback
- 1.3.4.3 Conduct Sprint Retrospective
- 1.3.4.4 Refine Product Backlog

**Expected Increment:** Working device fingerprinting and trusted-device verification integrated with authentication.

## **1.4 Sprint 4 – Leave Request and Validation**

### **1.4.1 Sprint Planning and Preparation**

- 1.4.1.1 Conduct Sprint Planning
- 1.4.1.2 Define Acceptance Criteria
- 1.4.1.3 Finalize Sprint Backlog

### **1.4.2 Leave Management User Stories**

- 1.4.2.1 Implement Leave Type Management
- 1.4.2.2 Implement Employee Leave Balances
- 1.4.2.3 Implement Leave Request Form
- 1.4.2.4 Implement Leave Request Submission
- 1.4.2.5 Implement Date Validation
- 1.4.2.6 Implement Weekday Calculation
- 1.4.2.7 Implement Overlapping Request Detection
- 1.4.2.8 Implement Leave Balance Validation

### **1.4.3 Testing and Integration**

- 1.4.3.1 Test Leave Request Submission
- 1.4.3.2 Test Date and Weekday Validation
- 1.4.3.3 Test Overlap Checking
- 1.4.3.4 Test Leave Balance Validation
- 1.4.3.5 Correct Identified Defects
- 1.4.3.6 Integrate Features with Previous Increments

## **1.4.4 Sprint Review and Retrospective**

- **1.4.4.1 Conduct Sprint Review**
- **1.4.4.2 Gather Stakeholder Feedback**
- **1.4.4.3 Conduct Sprint Retrospective**
- **1.4.4.4 Refine Product Backlog**

**Expected Increment:** Working leave request and validation features integrated with the existing system.

# **1.5 Sprint 5 – Leave Approval Workflow**

## **1.5.1 Sprint Planning and Preparation**

- **1.5.1.1 Conduct Sprint Planning**
- **1.5.1.2 Define Acceptance Criteria**
- **1.5.1.3 Finalize Sprint Backlog**

## **1.5.2 Approval User Stories**

- **1.5.2.1 Implement Leave Reservation**
- **1.5.2.2 Implement Supervisor Approval**
- **1.5.2.3 Implement HR Approval**
- **1.5.2.4 Implement Role-Based Approval Routing**
- **1.5.2.5 Implement Approval Status Tracking**
- **1.5.2.6 Implement Approval Notifications**

## **1.5.3 Testing and Integration**

- **1.5.3.1 Test Supervisor Approval**
- **1.5.3.2 Test HR Approval**
- **1.5.3.3 Test Approval Routing**
- **1.5.3.4 Test Approval Status Tracking**
- **1.5.3.5 Correct Identified Defects**
- **1.5.3.6 Integrate Approval Workflow with Leave Requests**

## **1.5.4 Sprint Review and Retrospective**

- **1.5.4.1 Conduct Sprint Review**
- **1.5.4.2 Gather Stakeholder Feedback**
- **1.5.4.3 Conduct Sprint Retrospective**
- **1.5.4.4 Refine Product Backlog**

**Expected Increment:** Working leave approval workflow with role-based approval routing.

## **1.6 Sprint 6 – Digital Signature-Based Approval**

### **1.6.1 Sprint Planning and Preparation**

- 1.6.1.1 Conduct Sprint Planning
- 1.6.1.2 Define Acceptance Criteria
- 1.6.1.3 Finalize Sprint Backlog

### **1.6.2 Digital Verification User Stories**

- 1.6.2.1 Implement WebAuthn/FIDO2 Passkey Verification
- 1.6.2.2 Implement Digital Signature Creation
- 1.6.2.3 Implement Digital Signature Verification
- 1.6.2.4 Integrate Digital Signature with Approval Workflow
- 1.6.2.5 Record Verified Approval

### **1.6.3 Testing and Integration**

- 1.6.3.1 Test Passkey Verification
- 1.6.3.2 Test Digital Signature Verification
- 1.6.3.3 Test Approval and Signature Integration
- 1.6.3.4 Correct Identified Defects

### **1.6.4 Sprint Review and Retrospective**

- 1.6.4.1 Conduct Sprint Review
- 1.6.4.2 Gather Stakeholder Feedback
- 1.6.4.3 Conduct Sprint Retrospective
- 1.6.4.4 Refine Product Backlog

**Expected Increment:** Working digitally verified approval process.

## **1.7 Sprint 7 – System Integration, Records, and Quality Improvement**

### **1.7.1 Sprint Planning and Preparation**

- 1.7.1.1 Conduct Sprint Planning
- 1.7.1.2 Define Acceptance Criteria
- 1.7.1.3 Finalize Sprint Backlog

## 1.7.2 Records and System Integration User Stories

- 1.7.2.1 Implement Approved Leave Balance Updates
- 1.7.2.2 Implement Rejected Leave Balance Restoration
- 1.7.2.3 Implement Leave History
- 1.7.2.4 Implement Audit Logging
- 1.7.2.5 Integrate All Completed System Features

## 1.7.3 System Testing and Defect Correction

- 1.7.3.1 Conduct Functional Testing
- 1.7.3.2 Conduct End-to-End Workflow Testing
- 1.7.3.3 Conduct Security Feature Testing
- 1.7.3.4 Identify Defects
- 1.7.3.5 Correct Identified Defects
- 1.7.3.6 Retest Corrected Features

## 1.7.4 Sprint Review and Retrospective

- 1.7.4.1 Conduct Sprint Review
- 1.7.4.2 Gather Stakeholder Feedback
- 1.7.4.3 Conduct Sprint Retrospective
- 1.7.4.4 Refine Product Backlog

**Expected Increment:** Integrated and tested LeaveSync system with leave records, balance updates, and audit logging.

## 1.8 System Development Tools and Technologies

The following technologies and tools were used in the development of LeaveSync:

| Category       | Tool / Technology                 |
|----------------|-----------------------------------|
| Backend        | PHP                               |
| Database       | MySQL                             |
| Frontend       | HTML5, CSS3, JavaScript           |
| Authentication | Multi-Factor Authentication (MFA) |

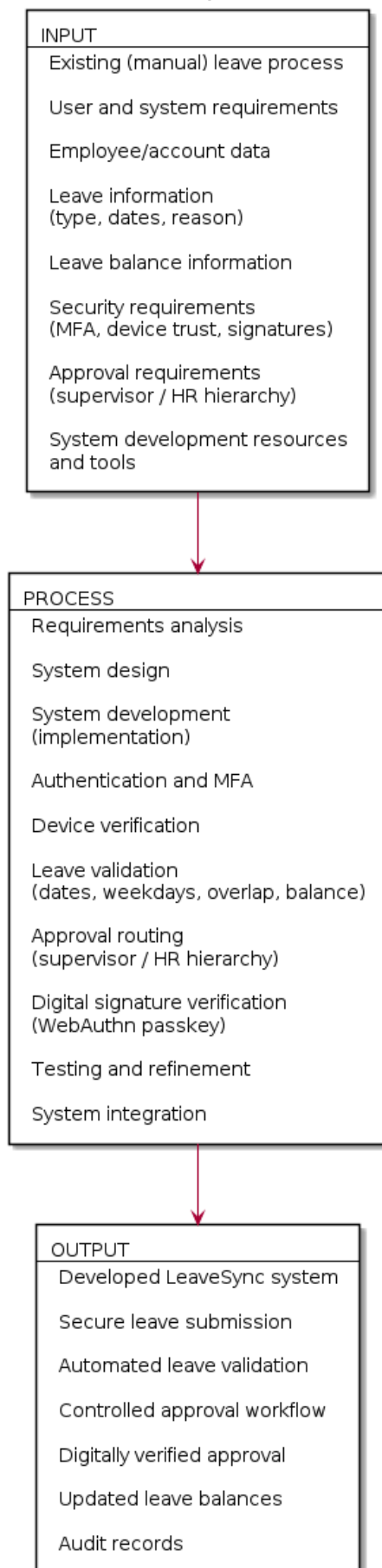
|                         |                                                        |
|-------------------------|--------------------------------------------------------|
| Device Security         | Device Fingerprinting                                  |
| Digital Signature       | WebAuthn/FIDO2 Passkeys and Cryptographic Verification |
| Cryptography            | OpenSSL, RSA, SHA-256                                  |
| Version Control         | Git and GitHub                                         |
| Diagramming             | PlantUML                                               |
| Development Environment | Visual Studio Code                                     |

These technologies were selected to support the functional requirements and security features of the LeaveSync system.

## 2. Conceptual Framework

The conceptual framework presents the study using the **Input-Process-Output (IPO) Model**. It shows how the existing situation, identified requirements, and development resources and tools (Input) are transformed through the activities of system development (Process) into the developed LeaveSync system and its resulting capabilities (Output).

## Conceptual Framework (Input-Process-Output Model)



## Figure 1. Conceptual Framework of LeaveSync Using the IPO Model

**Input.** The input consists of the existing manual leave process used as the baseline for comparison, the user and system requirements identified through interviews and observation, employee and account data, leave information (type, dates, and reason), leave balance information, the security requirements of the system (multi-factor authentication, device trust, and digital signatures), the approval requirements defined by the supervisor and HR hierarchy, and the development resources and tools needed to build the system.

**Process.** The process represents the requirements analysis, system design, and system development activities carried out to build LeaveSync. This includes implementing authentication and MFA, device verification, leave validation (checking the date range, counting only weekdays, rejecting overlapping requests, and confirming available balance), approval routing through the supervisor and HR hierarchy, digital signature verification through WebAuthn passkey assertions, and the testing, refinement, and integration of these components into a single working system.

**Output.** The output is the developed **LeaveSync system** and the capabilities it provides, including secure leave submission, automated leave validation, a controlled and digitally verified approval workflow, automatically updated leave balances, and an audit trail of important leave transactions and system activities.

## 3. Analysis of the Existing System

### 3.1 Process Name

#### Manual Leave Request and HRIS Approval Process

This is the existing leave management process used before the implementation of LeaveSync.

### 3.2 Objective

The existing process allows an employee to submit a leave request through Google Forms. The supervisor reviews the request and manually records the approval or rejection in the Human Resource Information System (HRIS).

### 3.3 Participants and Systems

- Employee
- Google Forms
- Supervisor
- HRIS (Human Resource Information System)

### 3.4 Inputs

- Employee information
- Leave type
- Leave start date
- Leave end date
- Leave reason

### 3.5 Outputs

- **Approved leave:** The approved leave is recorded in the HRIS and deducted from the employee's leave balance.
- **Rejected leave:** The rejected leave is recorded in the HRIS, and the employee's leave balance remains unchanged.

### 3.6 Business Rules of the Existing Process

1. The employee submits a leave request through Google Forms.
2. Google Forms records the submitted request and notifies the supervisor.
3. The supervisor reviews the submitted leave request.
4. The supervisor either approves or rejects the request.
5. The supervisor manually records the decision in the HRIS under the employee's record.
6. The HRIS deducts the corresponding leave days from the employee's balance when the request is recorded as approved.
7. A rejected request does not deduct leave days from the employee's leave balance.

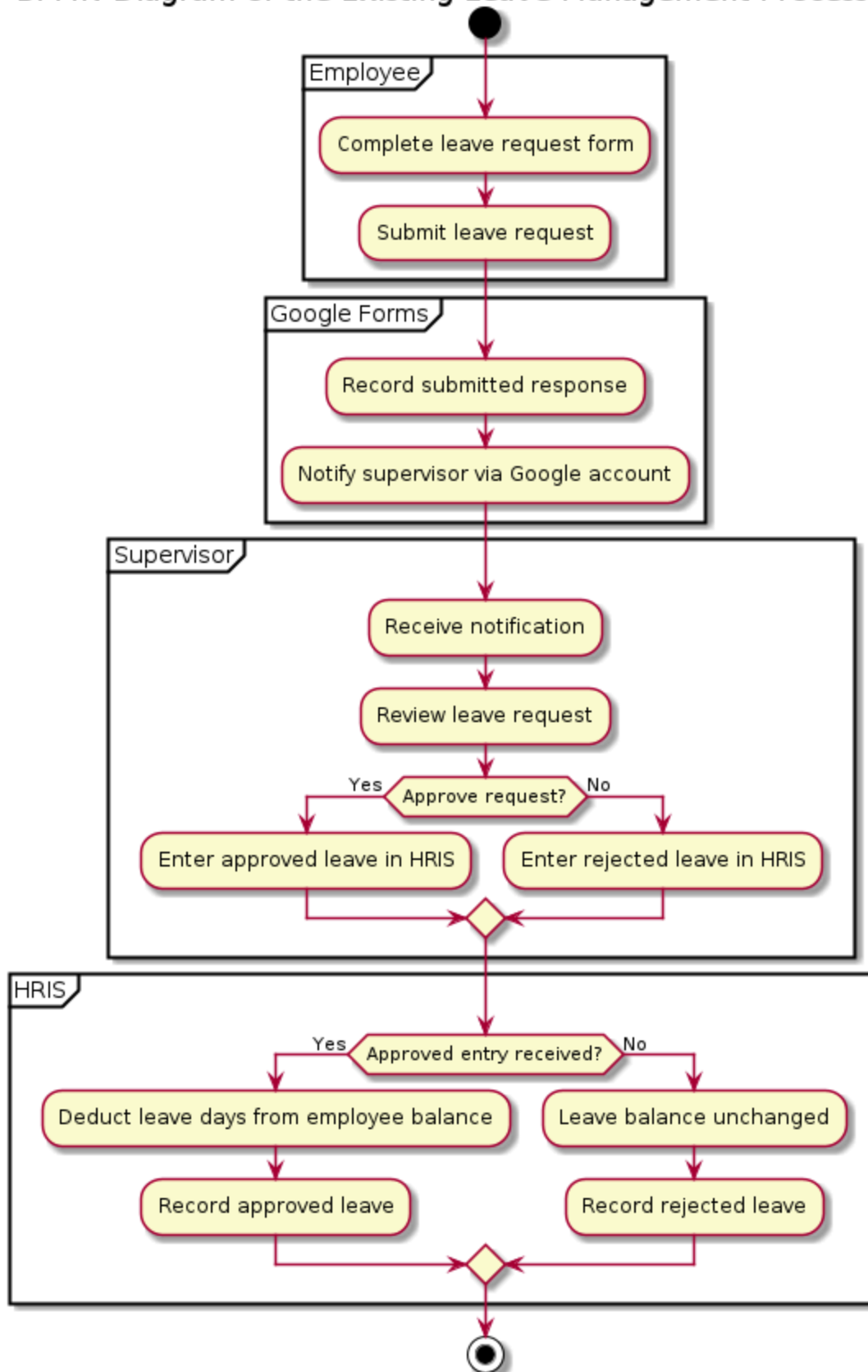
### 3.7 Weaknesses Identified in the Existing Process

- **No dedicated device or identity verification** — The process does not provide an additional mechanism for verifying that the request is being submitted from a trusted device belonging to the employee.
- **No digital signature for approval** — The approval is recorded through manual data entry in the HRIS and does not include a digital signature for verifying the authenticity of the approval.
- **Manual data entry** — The supervisor must manually transfer the decision from the submitted request to the HRIS, which may result in encoding errors or missed updates.
- **Separate systems** — Leave submission, notification, approval, and official record keeping are handled through separate tools rather than one integrated workflow.
- **Limited audit information** — The process does not provide a single consolidated record of the complete leave request and approval activities.
- **Manual leave balance checking** — The employee's available leave balance must be checked separately during the processing of the request.

### 3.8 BPMN Diagram of the Existing System

**Figure 2. BPMN Diagram of the Existing Leave Management Process**

## BPMN Diagram of the Existing Leave Management Process



### 3.9 BPMN Diagram Explanation

The BPMN diagram represents the complete existing leave management process involving the **Employee, Google Forms, Supervisor, and HRIS**.

**Employee.** The process begins when the employee enters the required leave information and submits the leave request through Google Forms.

**Google Forms.** Google Forms receives and records the employee's submitted information. It then provides a notification to the supervisor that a leave request has been submitted.

**Supervisor.** The supervisor receives the notification and reviews the submitted leave request. The supervisor then decides whether to approve or reject the request.

**Approval decision.** The approval decision is represented by an exclusive gateway because the request follows only one of two possible paths: **[Approved]** or **[Rejected]**.

**Approved path.** If the request is approved, the supervisor manually enters the approved leave information into the HRIS. The HRIS records the approved leave and deducts the corresponding leave days from the employee's leave balance.

**Rejected path.** If the request is rejected, the supervisor records the rejection in the HRIS. The HRIS records the rejected request without deducting leave days from the employee's balance.

**End of process.** The process ends after the HRIS records the final decision and performs the corresponding leave balance action.

### 3.10 Analysis of the Existing BPMN Process

The BPMN diagram shows that the existing leave management process is performed across several participants and systems. The employee submits the request through Google Forms, the supervisor reviews and decides on the request, and the HRIS serves as the official system for recording the result and updating the leave balance.

The process contains a manual transfer of information from the leave request to the HRIS. The use of separate systems for submission, notification, approval, and record keeping also means that the process is not handled through one integrated workflow.

The analysis of the existing process identified several areas that can be improved, including identity and device verification, approval authentication, automated validation, leave balance checking, approval routing, and audit recording. These identified requirements serve as the basis for the proposed LeaveSync system.

# 4. New System Process Using UML Activity Diagram

## 4.1 Process Name

**Submit and Process Leave Request in LeaveSync**

## 4.2 Objective

The process allows an authenticated employee to submit a leave request through LeaveSync. The system validates the request, checks the employee's leave balance and existing requests, routes the request to the appropriate approver, and updates the leave record based on the approval result.

## 4.3 Inputs

- Employee account
- Trusted-device information
- Leave type
- Leave start date
- Leave end date
- Leave reason

## 4.4 Outputs

- Approved leave request with updated leave balance
- Rejected leave request with the reserved leave days returned to the available balance
- Updated leave request status
- Recorded audit activity

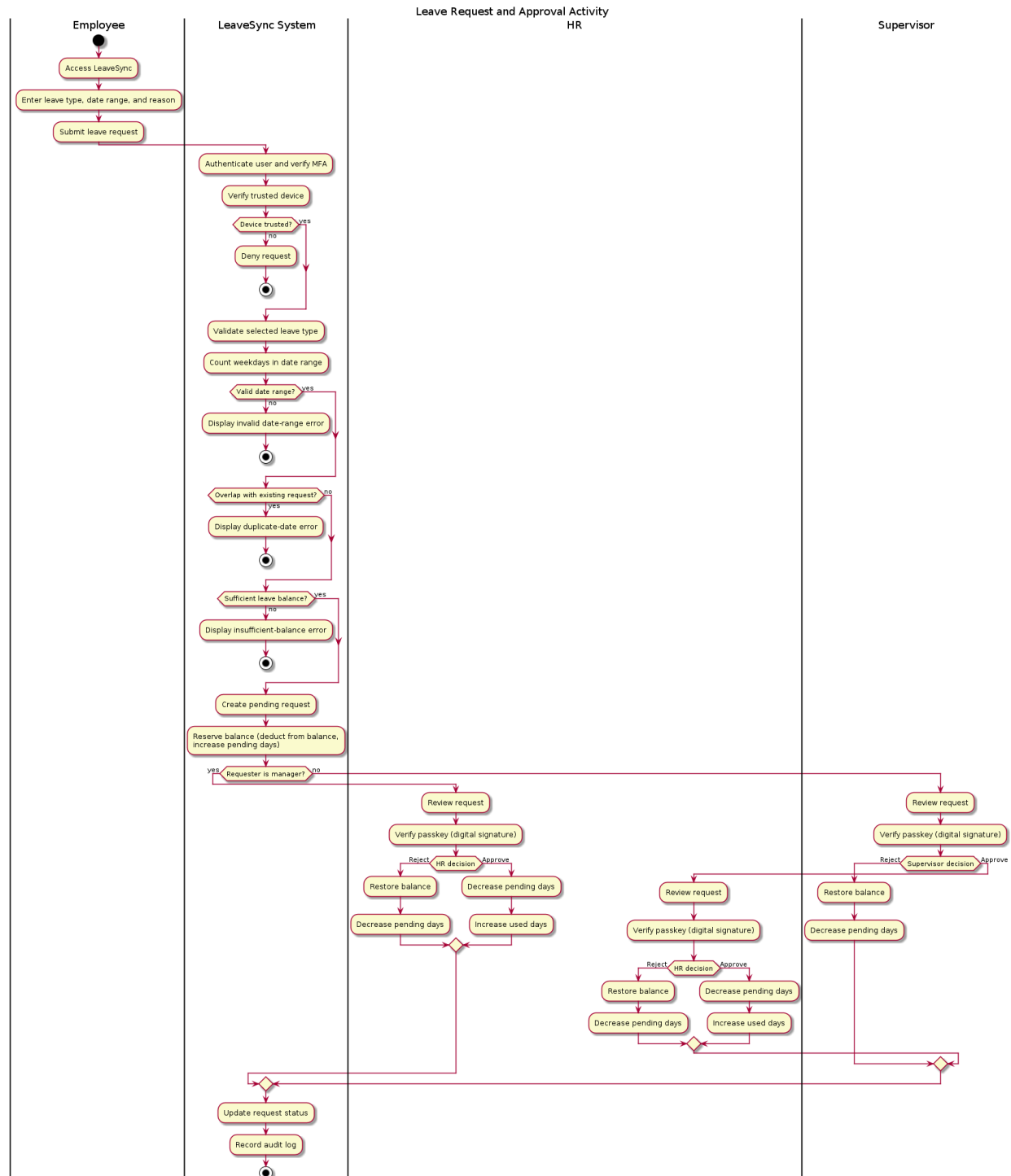
## 4.5 System Rules

1. The employee must be authenticated before submitting a leave request.
2. The employee must use a registered and trusted device.
3. The selected leave type must be available to the employee.
4. The end date must not be earlier than the start date.
5. Only weekdays are counted as leave days.
6. A date range containing no weekdays cannot be submitted.
7. A new request must not overlap an existing pending or approved request of the same employee.
8. The selected leave type must have sufficient available balance.
9. A regular employee's request is routed to the assigned supervisor before HR approval.

10. A manager's request is routed directly to HR.
11. Approval actions require passkey verification.
12. An approved request transfers the reserved leave days to used leave days.
13. A rejected request returns the reserved leave days to the available balance.
14. The system records the request and approval activities in the audit log.

## **4.6 UML Activity Diagram of the New System**

**Figure 3. UML Activity Diagram of the Proposed Leave Request and Approval Process**



## 4.7 UML Activity Diagram Explanation

The UML Activity Diagram describes how the LeaveSync software behaves when processing a leave request. It focuses on the actions, logical decisions, validation, data processing, approval routing, and successful or exceptional outcomes of the software process.

**Initial Node.** The solid black circle indicates that the activity begins when the employee accesses the LeaveSync leave request function. The initial node identifies where control enters the activity and does not represent an activity itself.

**Enter Leave Details.** The employee enters the required leave information, including the leave type, start date, end date, and reason. These values become the input data used by the system during validation.

**Submit Leave Request.** The employee submits the completed leave request to LeaveSync.

**Authenticate Employee.** LeaveSync verifies the employee's account before processing the request. If authentication fails, the system does not continue with the leave request process.

**Verify Trusted Device.** The system verifies whether the employee is accessing LeaveSync using a registered and trusted device. If the device is not trusted, the system displays an access or verification error and the activity ends.

**Validate Leave Type and Dates.** The system verifies that the selected leave type is available to the employee and checks whether the submitted dates are valid. The system also calculates the number of weekdays between the start and end dates. Saturdays and Sundays are excluded from the leave-day calculation.

The validation produces a decision:

- **[Valid]** → Continue processing.
- **[Invalid]** → Display validation error and end the request process.

**Check for Overlapping Request.** The system checks whether the employee has an existing pending or approved leave request covering any of the requested dates.

The decision produces two guarded outcomes:

- **[Overlap exists]** → Display overlapping-request error and end the process.
- **[No overlap]** → Continue processing.

Rejected or cancelled requests do not block a new request because they are no longer active leave requests.

**Check Leave Balance.** The system checks whether the employee has enough available balance for the selected leave type.

The decision produces two guarded outcomes:

- **[Insufficient balance]** → Display insufficient-balance message and end the process.
- **[Sufficient balance]** → Continue processing.

**Create Leave Request.** After successful validation, LeaveSync creates the leave request using the submitted leave information and calculated number of leave days.

**Reserve Leave Balance.** The system reserves the required number of leave days by reducing the available balance and adding the same number to the pending balance. This prevents the reserved days from being used by another leave request while the current request is awaiting approval.

**Determine Approval Route.** The system determines the appropriate approval route based on the employee's role.

The decision produces two guarded outcomes:

- **[Regular employee]** → Route request to assigned supervisor.
- **[Manager]** → Route request directly to HR.

**Review Leave Request.** The appropriate approver reviews the submitted leave request.

**Verify Passkey.** Before an approval or rejection is recorded, LeaveSync verifies the approver's passkey assertion.

The decision produces two guarded outcomes:

- **[Verification successful]** → Continue to approval decision.
- **[Verification failed]** → Do not record the decision and return to the appropriate approval action.

**Approve or Reject Request.** The authorized approver decides whether to approve or reject the request.

For a **regular employee**, supervisor approval is followed by HR approval. If the supervisor rejects the request, the request is marked rejected and the reserved balance is returned. If the supervisor approves the request, the request is routed to HR for final approval.

For a **manager**, the request proceeds directly to HR.

**Update Leave Balance.** When the final approver approves the request, LeaveSync transfers the reserved leave days from pending days to used days. When the request is rejected, the reserved leave days are returned to the available balance.

**Update Request Status.** The system updates the leave request status to reflect the final decision.

**Record Audit Activity.** LeaveSync records the relevant request and approval activity in the audit log, including the action performed and the corresponding user and time information.

**Activity Final Node.** The bull's-eye symbol indicates that the complete leave request activity has ended. The process terminates after the request has been successfully approved, rejected, or stopped because of an unsuccessful validation or verification condition.

## 4.8 Diagram Selection Rationale

The **BPMN diagram** is used for the existing system because it represents the complete organizational leave process involving people, systems, information, and business activities. It shows how the organization currently handles a leave request from submission through recording of the final decision.

The **UML Activity Diagram** is used for the proposed LeaveSync process because it focuses on how the software behaves when processing a leave request. It shows the system's actions, validation rules, decisions, data processing, approval routing, error conditions, and successful outcomes.

Therefore, the **BPMN diagram models how the organization delivers the existing leave process**, while the **UML Activity Diagram models how the LeaveSync software behaves within the proposed leave process**.